Movie Recommendation System

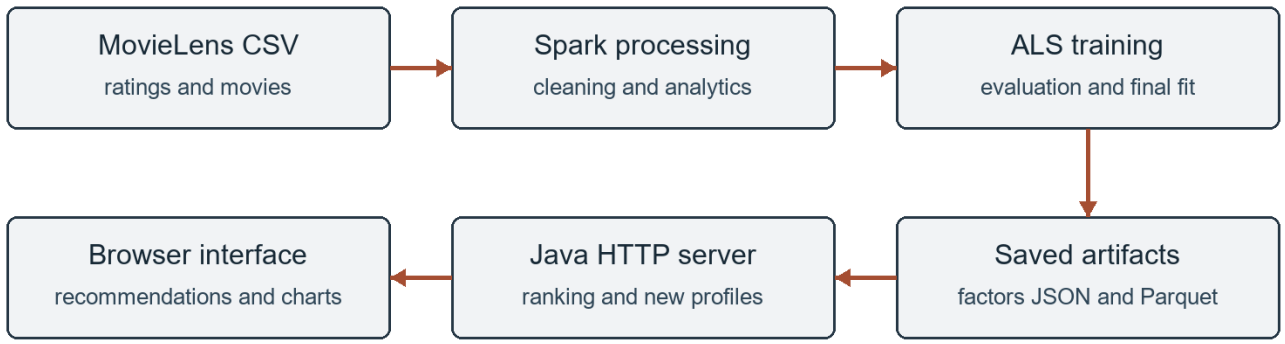
Project structure and technical explanation

This guide explains the submitted project as it is implemented. Use it to understand the code, run the application, and answer questions during the seminar.

What the application does

- Select an existing MovieLens user or enter ratings for a new profile.
- Get unseen-movie suggestions ordered by a preference score.
- Filter by genre, number of results and minimum rating count.
- View rating statistics and the saved evaluation results.

Overall flow



Two separate stages

Stage	Command	Work performed
Batch training	train.cmd	Read CSV, clean records, analyse ratings, train ALS, evaluate and export files.
Application serving	run.cmd	Load saved factors, handle browser requests and rank movies. Spark is not started for each request.

Reading order

Pages	Topic
2-3	Folders, Java classes and data preparation
4-5	ALS, score calculation, new users and filters
6-7	Saved data, HTTP endpoints and browser behaviour
8-9	Evaluation, tests, setup and troubleshooting
10-12	Big Data concepts, viva questions and raw source excerpts

Project folder: C:/Users/Admin/Desktop/Codes/BDA-Movie-Recommender

Repository: https://github.com/php2k6/bda-proj

1 Folders and Java classes

```
BDA-Movie-Recommender/
  src/main/java/college/bda/   Application and training code
  src/main/resources/          Logging configuration
  src/test/java/college/bda/   Java unit tests
  web/                         HTML, CSS and JavaScript
  data/ml-latest-small/        Original MovieLens files
  artifacts/                   Model, statistics and Parquet data
  scripts/                     PowerShell and HDFS helpers
  sql/                         SQL and optional database examples
  dist/                        Executable application JAR
  report/                      College report
  docs/                        Screenshots and viva notes
  pom.xml                      Java dependencies and build
  setup.cmd / setup.sh         Install dependencies and build
  train.cmd / train.sh         Process data and train
  run.cmd / run.sh             Start the application
```

Class	Responsibility
Main.java	Parses the command and --name value options. Calls download, train or serve. Defaults to serve.
TrainPipeline.java	Runs Spark CSV preparation, RDD counts, SQL statistics, holdout evaluation and final training.
Recommender.java	Loads factors; searches, scores and filters movies; fits a new profile.
ModelData.java	Java records for Rating, Movie and Recommendation, plus maps of user/movie factor vectors.
WebServer.java	JDK HTTP server, local static files, JSON requests and responses.
DataDownloader.java	Official MovieLens download, safe ZIP extraction and SHA-256 calculation.
JavaLocalFileSystem.java	Java NIO adapter for local Spark file operations on Windows.

Build and runtime details

- Java 17 compiles and runs the application. Maven reads pom.xml and downloads the pinned libraries.
- Spark SQL and MLlib 3.5.7 use Scala 2.12 libraries through the Java API. The application code is Java.
- Jackson converts model objects to JSON and reads JSON files and request bodies.
- The small distribution JAR includes project classes and Jackson. Training uses the additional Spark libraries in target/dependency.
- Main invokes training reflectively, so serving can run without loading Spark classes. The website files and artifacts still need to stay beside the JAR.

2 Dataset and preparation

Source: GroupLens MovieLens latest small. The supplied files contain 100,836 ratings, 610 anonymous users and 9,742 catalogue movies.

File	Fields	Used for
ratings.csv	userId, movieId, rating, timestamp	Preference data and user histories
movies.csv	movieId, title, genres	Titles, genre filters and catalogue joins
tags.csv / links.csv	Original MovieLens metadata	Included, but not used by this model

```
ratings.csv:
userId,movieId,rating,timestamp
1,1,4.0,964982703

movies.csv:
movieId,title,genres
1,Toy Story (1995),Adventure|Animation|Children|Comedy|Fantasy
```

- The sample means User 1 rated Toy Story with 4.0 stars. movieId links the rating to the catalogue.
- timestamp is Unix time in seconds. It controls rating order during splitting and determines the Parquet year partition.
- A movie can belong to several genres. The application splits the genre string on the vertical bar.

Cleaning sequence in TrainPipeline

- Read CSV with headers, then cast columns to the required numeric types.
- Require positive 32-bit integer IDs. Ratings must be finite values from 0.5 to 5, in half-star steps; timestamps must be nonnegative.
- For each duplicate user/movie pair, keep the latest timestamp; equal timestamps use the higher rating.
- Keep catalogue entries with a valid ID and nonempty title. Deduplicate movie IDs.
- Join ratings to the known movie IDs. Remove rating rows for movies absent from the catalogue.
- Persist frequently reused DataFrames with MEMORY_AND_DISK.

What was observed

Check	Result
Invalid / duplicate / unknown-movie rating rows removed	0 / 0 / 0
Cleaned ratings retained	100,836
Average dataset rating	3.502
Most frequent rating	4.0, occurring 26,818 times

The original dataset already passes these checks. Cleaning rules are retained so another input file can be validated in the same way. Missing ratings are not filled with zero: an unobserved preference is not a recorded dislike.

3 How ALS learns preferences

The rating matrix

- Think of rows as users, columns as movies, and recorded ratings as filled cells.
- Most cells are empty because each user has rated only a fraction of the catalogue.
- Collaborative filtering learns patterns across users and movies from the observed ratings.

Latent factor vectors

- ALS means Alternating Least Squares. Each user and movie receives a vector of learned numbers.
- The real model uses 12 factors. They are learned numerical dimensions, not manually named genres.
- A user/movie score is the dot product of their vectors. It estimates preference for that user.

```
score(u, m) = x[u][0] * y[m][0]
            + x[u][1] * y[m][1]
            + ...
            + x[u][11] * y[m][11]
```

Small worked example

These two-factor values are illustrative; they are not the trained factors.

Vector	Factor 1	Factor 2
User	0.8	0.4
Movie A	1.0	0.5
Movie B	0.2	0.9

- Movie A: (0.8 × 1.0) + (0.4 × 0.5) = 1.00.
- Movie B: (0.8 × 0.2) + (0.4 × 0.9) = 0.52.
- Movie A ranks above Movie B, unless it is excluded by history or filters.

Alternating updates

- Hold movie factors fixed and solve for user factors.
- Hold user factors fixed and solve for movie factors.
- Repeat updates for the configured iterations. The objective combines squared rating error with regularisation.
- Regularisation discourages excessively large factors. Spark ALS-WR scales regularisation by the number of observations for each user or movie.

Setting	Value	Meaning
rank	12	Number of factors
maxIter	10	Alternating update iterations
regParam	0.1	Regularisation strength
seed	42	Initialisation seed
implicitPrefs	false	Use explicit star ratings

Settings were fixed for this demonstration. No parameter search or tuning on the test set is claimed. Scores are unconstrained estimates, so they can exceed 5; they are not star ratings or probabilities.

4 Ranking and new users

Existing-user request

- Look up the user factor vector in model.users.
- Build a HashSet of every movie ID in that user's saved history.
- Scan the catalogue and exclude rated movies and movies below the minimum rating count.
- Apply an exact genre membership check. For a known user, skip movies without learned factors.
- Calculate the factor dot product; keep finite scores.
- Sort by descending score, then descending rating count, title and movie ID. Return up to the requested limit.

New profile calculation

- A profile contains 1-30 different movies with ratings in half-star steps from 0.5 to 5.
- Validate movie IDs, learned factors, duplicates and numeric rating values.
- Keep movie factors fixed and solve for one new user vector. The entire ALS model is not retrained.

```
(Y^T Y + lambda * n * I) x = Y^T r
```

Symbol	Meaning
Y	Vectors of the movies selected by the user
r	Entered ratings
n	Number of entered ratings
lambda	Saved regularisation value, 0.1
x	New user vector to calculate
I	Identity matrix

- The code accumulates a 12 × 12 Gram matrix and a 12-value right-hand-side vector.
- It adds lambda × n to the diagonal, then solves using pivoted elimination.
- The resulting vector goes through the same ranking and filtering method as an existing user.
- Selected movies are excluded. The new profile does not change the CSV files or stored histories.

Unknown-user fallback

```
weightedRating = (count * movieAverage + 50 * globalMean)
                 / (count + 50)
```

An unknown user ID has no saved vector. The API then ranks by this popularity score. A small rating count pulls the result toward the global mean, reducing the influence of a film with only a few high ratings.

Genre is a filter in this implementation, not a feature used to train ALS. The API accepts up to 50 results; the UI offers 5, 10 or 20. If no candidates match, it returns an empty list.

5 Saved files and data structures

Location	Contents	Read by
artifacts/model.json	Factors, movie catalogue and user histories	Recommender
artifacts/analytics.json	Dataset counts, distribution, genre and popularity statistics	Summary API and charts
artifacts/metrics.json	Split counts, errors, coverage, settings, hashes and runtime	Summary API and report
artifacts/recommendations.ndjson	One precomputed top-10 document per user	Optional document database exercise
artifacts/parquet/ratings	Cleaned rating columns, partitioned by year	Spark SQL / optional Hive
artifacts/parquet/movies	Cleaned movie catalogue	Spark SQL / optional Hive

ModelData.java

```
rank          -> factor count
regParam      -> regularisation value
globalMean    -> dataset mean rating
users         -> user ID  -> factor vector
items         -> movie ID -> factor vector
histories     -> user ID  -> list of Rating(movieId, rating)
movies        -> list of Movie records
```

- Movie stores movieId, title, genres, ratingCount, averageRating and weightedRating.
- Recommendation returns the title, genres, support, average and personalised score.
- Jackson serialises these Java structures into JSON. Integer map keys appear as strings in JSON objects.

Why Parquet is included

- Parquet stores typed data by column. Queries can read required columns rather than full CSV text.
- Partitioning ratings by year lets year-filtered queries skip other year folders.
- The pipeline reloaded the Parquet output and verified that all 100,836 cleaned rating rows remained.

Why a JSON recommendation export is included

One document groups a user ID with an array of recommendations. That is a document-oriented schema suitable for a NoSQL database. The local app does not query MongoDB; the supplied import example is optional.

Scale limit of this export

- Training collects factor tables, catalogue statistics and histories into driver memory.
- Serving scans the in-memory catalogue for each request. This is practical for the sample; much larger data needs shared storage and a different serving/indexing design.
- Serving uses factor JSON, not a loaded Spark ALSModel. A running Spark session is therefore unnecessary for normal browser use.

6 Server, APIs and browser

WebServer uses Java's built-in HttpServer on 127.0.0.1:8080 with a fixed pool of four request threads. Jackson writes JSON responses. The server is local to the computer.

Method and route	Purpose
GET /	Serve web/index.html
GET /app.js and /style.css	Serve the two static browser assets
GET /api/summary	Return analytics, metrics and available genres
GET /api/users	Return user IDs and history sizes
GET /api/movies?q=matrix	Search titles, returning up to 20 matches
GET /api/recommend	Recommend for an existing or unknown user ID
POST /api/profile	Recommend using ratings supplied in a JSON body

Existing-user request

```
GET /api/recommend?user=1&limit=10&genre=Drama&minimum=20
```

- user: MovieLens user ID; limit: maximum results; genre: optional filter; minimum: required rating count.
- The response contains method, userId, ratedCount and a recommendations array.
- Each suggestion includes movieId, title, genres, ratingCount, averageRating and score.

New-profile request

```
POST /api/profile
Content-Type: application/json

{"ratings":[{"movieId":1,"rating":4.5},
            {"movieId":260,"rating":5.0}],
 "limit":10,"genre":"","minimum":20}
```

Browser responsibilities

- index.html defines controls and result tables. style.css controls layout. app.js handles events and fetch requests.
- On load, fetch summary and users, populate the controls, then request recommendations.
- Movie search starts after two typed characters and a short debounce. Matching titles are added to a temporary JavaScript ratings array.
- New ratings are not saved to a database or localStorage. Reloading the page clears the profile.

Errors and validation

Invalid inputs return HTTP 400; unsupported methods return 405; unknown routes return 404. Profile request bodies are limited to 16 KB. Static file routes are explicitly listed, and JSON responses use no-store caching. The UI displays a request error instead of silently ignoring it.

7 Evaluation and tests

How the split is constructed

- For each user, sort ratings by timestamp, then movie ID.
- Use the earliest floor(80%) for training, retaining at least one training row. Use the remaining rows for testing.
- Train an evaluation ALS model on the training partition only.
- Spark coldStartStrategy="drop" removes test rows lacking required user/movie factors.
- Measure both ALS and a training-global-mean baseline on those same scored rows.
- Finally, fit a different serving model on all cleaned ratings. Its factors power the app.

Measurement	Observed value
Training / test ratings	80,419 / 20,417
Scored / excluded test rows	18,715 / 1,702
Prediction coverage	91.66%
ALS RMSE / MAE	0.9316 / 0.7176
Global-mean baseline RMSE	1.0569
Spark master	local[2]

Meaning of the measurements

```
error = predictedRating - actualRating
MAE   = mean(abs(error))
RMSE  = sqrt(mean(error * error))
coverage = scoredTestRows / allTestRows * 100
```

- Lower error is better. RMSE penalises large errors more strongly than MAE.
- Coverage prevents the excluded cold-start rows from being hidden behind a single error figure.
- Raw predictions are not clipped. Neither RMSE nor MAE is an accuracy percentage.
- Lower rating error does not establish top-10 ranking quality. Precision and recall at 10 were not measured.

Verification performed

- All 16 Java unit tests passed: ordering, history exclusion, support/genre filters, fallback, input rejection, profile fitting and history preservation.
- The profile solver was checked against a small analytic solution.
- The full Spark pipeline and Parquet read-back passed. API and browser checks covered existing users, new profiles, search, filters and validation errors.

Important limitation: this is a per-user temporal split, not one global time cutoff. Another user’s later ratings may appear in training. Results describe this experiment, not a strict future-time deployment test.

8 Setup, running and troubleshooting

Windows commands

```
cd C:\Users\Admin\Desktop\Codes\BDA-Movie-Recommender
setup.cmd
run.cmd

REM Reprocess the CSV files and train again:
train.cmd

REM Use another port:
run.cmd --port 8081
```

Open http://127.0.0.1:8080. Keep the server terminal running. Ctrl+C stops it. REM lines are comments in Command Prompt.

What setup does

- Find JDK 17, or install Eclipse Temurin 17 through winget when it is missing.
- Download a private Maven 3.9.11 installation and verify its SHA-256 checksum.
- Build the application, run Java tests and copy training dependencies into target/dependency.
- Copy the executable JAR into dist; download MovieLens only if required.
- Train if the saved model is missing. Included artifacts allow the prepared model to be reused.

Linux and macOS

```
bash setup.sh
bash run.sh
bash train.sh
```

Setup supports apt on Debian/Ubuntu and an existing Homebrew installation on macOS. Other systems need JDK 17, curl and unzip installed first. Full execution was tested on Windows; Linux/macOS scripts were syntax-checked only.

Useful training options

```
train.cmd --rank 8 --iterations 10 --regularization 0.1   --output experiments/rank8
```

Type the example as one line. --input selects a folder containing both CSV files. --output must stay inside the project. Use a separate output folder when comparing experiments.

Problem	What to check
Port already in use	Use another --port value or stop the previous server.
Model not found	Keep artifacts beside the JAR; run setup.cmd or train.cmd.
Java or training failure	Use JDK 17 and the provided scripts, including Java module-opening options.
No matching movies	Lower the minimum rating count or change the genre.
Profile rejected	Use 1-30 unique known movies and valid half-star ratings.
Dependency download failure	Check Internet access and retry setup; do not disable certificate validation.

Allow about 4 GB RAM and 2 GB free disk space for setup/training. Training scripts use a 2 GB Java heap. Optional native Hadoop/BLAS warnings occurred in the tested run; the Java fallbacks still completed the pipeline.

9 Big Data concepts and viva notes

Concept	Where it appears
Structured / semi-structured data	CSV tables; JSON model and recommendation documents
Volume	100,836 ratings in an educational sample
Velocity	Batch input only; no streaming updates implemented
Variety	CSV input, JSON export, Parquet storage
Veracity	Validation, deduplication and catalogue joins
RDD transformations / actions	mapToPair and reduceByKey; count executes the calculation
DataFrames / SQL	Typed filtering, windows, grouping and joins
Persistence	MEMORY_AND_DISK for reused DataFrames
MLlib	ALS collaborative filtering
HDFS / Hive / NoSQL	Optional examples; services are not running in the local app

Questions you should be ready to answer

- Why Spark for a small dataset? To demonstrate processing and MLlib APIs on a dataset that can be run and explained on one computer.
- What is lazy evaluation? Transformations describe work; actions such as count trigger execution.
- What is rank? The length of the user/movie vectors, set to 12.
- Does genre train the model? No. It is a result filter; ALS learns from user/movie ratings.
- Why can scores exceed 5? The factor dot product has no fixed upper bound.
- Does a new user retrain Spark? No. One user vector is fitted using the fixed movie factors.
- Where is the database? The tested app uses files and in-memory objects; JSON database import is optional.
- Can this run on billions of ratings unchanged? No. Collecting histories/factors and scanning the catalogue limit the current export and server.

What to improve next

- Use a global chronological split and a separate validation set.
- Measure precision, recall and coverage for top-10 recommendation lists.
- Use movie metadata for films with no ratings, and add persistent user profiles if required.
- For larger deployments, use shared output storage, distributed preparation and a serving design that avoids full in-memory catalogue scans.

Source references

Implementation: src/main/java/college/bda, web, scripts and pom.xml. Measured values: artifacts/metrics.json and analytics.json. Tests: src/test/java/college/bda/RecommenderTest.java.

GroupLens MovieLens: https://grouplens.org/datasets/movielens/latest/ Dataset conditions: data/ml-latest-small/README.txt Harper and Konstan (2015): https://doi.org/10.1145/2827872 Spark ALS: https://spark.apache.org/docs/3.5.7/ml-collaborative-filtering.html

10 Raw source code

```
ALS als = new ALS().setUserCol("userId").setItemCol("movieId").setRatingCol("rating")
        .setRank(rank).setMaxIter(iterations).setRegParam(regularization).setSeed(42L)
        .setNumUserBlocks(4).setNumItemBlocks(4).setColdStartStrategy("drop").setImplicitPrefs(false);
System.out.println("Training evaluation model on each user's earliest 80% of ratings...");
ALSModel evalModel = als.fit(train);
Dataset<Row> predictions = evalModel.transform(test).persist(StorageLevel.MEMORY_AND_DISK());
long scored = predictions.count();
if (scored == 0) throw new
    IllegalArgumentException("No holdout records have known user and movie factors.");
RegressionEvaluator evaluator = new
    RegressionEvaluator().setLabelCol("rating").setPredictionCol("prediction");
double rmse = evaluator.setMetricName("rmse").evaluate(predictions);
double mae = evaluator.setMetricName("mae").evaluate(predictions);
double trainMean = ((Number)train.agg(avg("rating")).first().get(0)).doubleValue();
double baseline = evaluator.setMetricName("rmse").evaluate(predictions.withColumn("prediction",
    lit(trainMean)));
```

```
public List<Recommendation> forUser(int id, int limit, String genre, int minimumRatings) {
    Set<Integer> seen = new HashSet<>();
    for (Rating r : model.histories.getOrDefault(id, List.of())) seen.add(r.movieId());
    return rank(model.users.get(id), seen, limit, genre, minimumRatings);
}
public List<Recommendation> forProfile(List<Rating> ratings, int limit, String genre, int
    minimumRatings) {
    validateRatings(ratings);
    Set<Integer> seen = new HashSet<>();
    for (Rating r : ratings) seen.add(r.movieId());
    return rank(fitProfile(ratings), seen, limit, genre, minimumRatings);
}
```

11 Raw source code continued

```
double[] fitProfile(List<Rating> ratings) {
    int k = model.rank;
    double[][] gram = new double[k][k];
    double[] rhs = new double[k];
    for (Rating r : ratings) {
        double[] y = model.items.get(r.movieId());
        for (int i = 0; i < k; i++) {
            rhs[i] += y[i] * r.rating();
            for (int j = 0; j < k; j++) gram[i][j] += y[i] * y[j];
        }
    }
    for (int i = 0; i < k; i++) gram[i][i] += model.regParam * ratings.size();
    return solve(gram, rhs);
}

private List<Recommendation> rank(double[] vector, Set<Integer> seen, int limit,
                                  String genre, int minimumRatings) {
    if (limit < 1 || limit > 50 || minimumRatings < 0 || minimumRatings > 10000)
        throw new IllegalArgumentException("Invalid result limit or minimum rating count.");
    List<Recommendation> result = new ArrayList<>();
    for (Movie movie : model.movies) {
        if (seen.contains(movie.movieId()) || movie.ratingCount() < minimumRatings) continue;
        if (genre != null && !genre.isBlank() &&
    !Arrays.asList(movie.genres().split("\\|")).contains(genre)) continue;
        double[] item = model.items.get(movie.movieId());
        if (vector != null && item == null) continue;
        double score = vector == null ? movie.weightedRating() : dot(vector, item);
        if (Double.isFinite(score)) result.add(new Recommendation(movie.movieId(), movie.title(),
    movie.genres(),
                movie.ratingCount(), movie.averageRating(), score));
    }
    result.sort(Comparator.comparingDouble(Recommendation::score).reversed()
            .thenComparing(Comparator.comparingLong(Recommendation::ratingCount).reversed())
            .thenComparing(Recommendation::title).thenComparingInt(Recommendation::movieId));
    return result.subList(0, Math.min(limit, result.size()));
}
static double dot(double[] a, double[] b) {
    double sum = 0;
    for (int i = 0; i < a.length; i++) sum += a[i] * b[i];
    return sum;
}
```